the link which we need to inject into the storyLink's `href` attribute. Similarly we get the story title and put that inside the link using the `innerHTML` property. Now we have a link to the feed story.

```
    story.appendChild(storyLink);
      document.body.
appendChild(story);
```

This newly created link is now appended inside the story `<div>` we created earlier. This complete link within a `<div>` is now appended to the document body. We skipped the `document.body.appendChild(document.createElement("hr"));` earlier because all it does is create a `<hr>` or a horizontal rule element which just serves to improve the looks of the extension by adding a line around every story.

All this is contained inside a `<script>` tag in the `popup.html` file. We will also add a few CSS instructions to improve the appearance of the popup.

```
  body {
   min-width:180px;
  overflow-x:hidden;
}
```

This ensures that the popup gets a width of at least 180 pixels, and any overflowing width is hidden.

options

Since we are just looking at the basic configuration scenario, here's the options.html file in its entirety:

```
  <html>
    <head>
      <title>Options for my
extension</title>
      <script>
        function main()
        {
          if(!window.
localStorage["numStories"]){
            window.
localStorage["numStories"] = 5;
          }
          document.
getElementById('numStories').
value = window.
localStorage["numStories"];
        }
        function store()
        {
          window.
localStorage["numStories"]
= document.
getElementById('numStories').
value;
        }
      </script>
    </head>
    <body onload="main()">
      <div>
        <h2>
          Options for my exten-
sion
        </h2>
        <label for="numStories"
>How many story links should be
displayed?</label>
        <input id="numStories"
type="text" />
        <input type="button"
onclick="store()" value="Set" />
      </div>
    </body>
  </html>
```

This is what you'd expect from pretty much any HTML file. Now let's delve in.

First inside our page `<body>` we have a text input element with an id of `numStories` which is asking the user how many stories need to be displayed from the feed. We also have an input button that will call the `store()` function when clicked to save the setting. Also, the `<body>` tag itself runs the `main()` function on loading.

Here the main function is checking if the `localStorage` value for `numStories` is set. In case the extension is being run for the first time, it might not be, in which case it is set to 5. Its value is then read and put in the `numStories` textbox.

The `store()` function reads the value from the text box and saves it in `localStorage`. The options page is done. However, it's useless if we don't use the option in our main popup page itself! So now we go about modifying that to use this configuration parameter. This can be very simply be achieved by modifying the for loop to look as follows:

```
    for (var i = 0; i<window.
localStorage["numStories"]; i++)
```

Although, before the `window.localStorage["numStories"]` value is used, you should also check if it exists, and set it to a constant value (such as 5) in case it doesn't.

Testing the extension

As said before, in Chrome you needn't restart the browser after each change. You can in fact load the directory containing the extension files as is, without packing into a CRX file. To go about testing your extension, open the Extensions page using the wrench menu. On the page that appears, you'll see a link titled `Developer mode` near the top-right corner of the page. This will reveal extended options among which will be the option to load an unpacked extension. Click on the button to `Load unpacked extension...` and point the folder browse dialog which results to the directory with the extension files.

You extension will now appear in the list, with the option to reload, disable and uninstall. The options button should also be enabled. You can reload the extension after making even a small change, and see how it affects your extension.

Conclusion

You can see how easy it is to get started with creating extensions for Chrome. However, we have only just scratched the surface. There's a lot more you can do with Chrome than just display a popup from a toolbar button. You can respond to event in the browser to trigger your own code. You can add content scripts which can change the very looks of a page. You can even pass messages between extensions to make them work together.

Remember that to make full use of this tutorial you should actually code the extension yourself while adding your own changes to the mix, instead of just reading it. Hopefully after you are done with this tutorial, you will explore further the intricacies of creating Chrome extensions.

With extensions, Chrome has made a significant step in gaining parity with Firefox, which still remains the favourite of those who wish to customize their browsing experience to the fullest. Hopefully we have added a few extension developer to the mix, who will push the envelope further.